PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
DUOC UC

DOCUMENTO DE COBERTURA DE TESTING

Sistema de Gestión y Venta de Gas

"Distribuidora de Gas El Volcán"

Asignatura	DSY1104 — Desarrollo Fullstack II
Integrantes	Nicolás Arancibia Pablo Gómez Benjamín González
Profesor(a)	Hernán Saavedra
Versión	2.1

CHILLÁN · REGIÓN DE ÑUBLE · CHILE

Documento de Cobertura y Análisis de Testing

Proyecto: Distribuidora de Gas El Volcán (migración a React) **Asignatura:** DSY1104 — Desarrollo Fullstack II **Evaluación:** Evaluación Parcial N° 2 **Herramientas de testing:** Vitest + React Testing Library **Fecha:** Octubre 2026

---

1. Objetivo

Este documento describe la estrategia de **pruebas unitarias** aplicada al frontend del proyecto React, presenta los **resultados de ejecución**, el **reporte de cobertura** obtenido y un **análisis** de los mismos.

Las pruebas se desarrollaron con **Vitest**, el framework de testing visto en clases para proyectos React + Vite, y cubren exactamente los tipos que pidió el docente en su retroalimentación:

- **Componentes que validan** (validaciones de formularios).
- **Formularios que interactúan con el usuario** (`<Login />`).
- **Compras / carritos** (lógica del carrito y cálculo de totales).
- **CRUD** (catálogo de productos: ediciones y creación desde el panel admin).
- Componentes con **props**, **estado** y **mocks** que aíslan la comunicación.

---

2. Herramientas y versiones

Herramienta	Rol	Versión
Vitest	Framework de pruebas (describe / test / expect / vi) integrado con Vite	3.2.7
jsdom	Entorno que simula el navegador (DOM, `localStorage`, eventos)	26.1.0
@testing-library/react	Renderizado de componentes y consultas del DOM (`render` , `screen`)	16.3.3
@testing-library/dom	API de consultas del DOM	10.4.2
@testing-library/jest-dom	Matchers de DOM (`toBeInTheDocument` , `toHaveTextContent` , `toHaveValue` , `toHaveAttribute` , …)	6.6.3
@testing-library/user-event	Simula la interacción real del usuario (escribir, hacer clic)	14.6.1
@vitest/coverage-v8	Instrumentación y reporte de cobertura con el motor **V8** de Node	3.2.4

Entorno de ejecución: Node.js v24 · Vitest 3.2.7 · Windows.

> **Nota de diseño:** Vitest reutiliza la configuración de **Vite** (`vite.config.js`) y el plugin de React. El entorno `jsdom` + `setupFiles` (que carga `@testing-library/jest-dom`) se declaran en el bloque `test` de esa misma configuración; la cobertura se genera con el **provider** `v8` , que mide la ejecución a nivel del motor de JavaScript.

---

3. Estructura de las pruebas

```
src/
├── setupTests.js                    # Config global: matchers de jest-dom
├── utils/
│   ├── validaciones.test.js         # 2 pruebas (lógica de formularios + helpers)
│   ├── carrito.test.js              # 3 pruebas (lógica del carrito / compras)
│   └── productos.test.js            # 1 prueba  (lógica del catálogo: CRUD)
├── components/
│   ├── Producto.test.jsx            # 1 prueba  (props + DOM + mock con vi.fn)
│   └── Navbar.test.jsx              # 1 prueba  (props + DOM)
└── pages/
    ├── Nosotros.test.jsx            # 1 prueba  (estado con useState)
    └── Login.test.jsx               # 1 prueba  (formulario interactivo con user-event)
```

Total: 10 pruebas unitarias (máximo indicado por el docente: 10).

Cómo ejecutar

```
npm install           # instala dependencias (incluidas las de testing)
npm test              # ejecuta las 10 pruebas una sola vez (modo CI)
npm run test:watch    # modo vigilancia durante el desarrollo
npm run test:coverage # ejecuta las pruebas y genera el reporte de cobertura
```

El reporte de cobertura se genera en:

- `coverage/index.html` — reporte navegable por archivo y línea.
- `coverage/lcov.info` — formato estándar (LCOV) para herramientas externas.

---

4. Inventario de las 10 pruebas

#	Archivo	Prueba	Qué verifica	Tipo
1	`utils/validaciones.test.js`	*valida campos simples: RUN, teléfono, correo y formato de precio*	Funciones puras `validarRun`, `validarTelefono`, `validarCorreo`, `correoDominioPermitido`, `formatearPrecio` con casos válidos e inválidos	Lógica (validación)
2	`utils/validaciones.test.js`	*validarFormularioRegistro, validarFormularioContacto y helpers de almacenamiento*	Devuelven el **primer error** de cada formulario; `leerLista` / `leerObjeto` (incluye JSON corrupto)	Lógica + `localStorage`

#	Archivo	Prueba	Qué verifica	Tipo
3	`utils/carrito.test.js`	*agregarItem e incrementarItem…*	Agrega/acumula cantidades y **no muta** el arreglo original	Lógica (carrito)
4	`utils/carrito.test.js`	*decrementarItem y eliminarItem…*	Descuenta, elimina al llegar a 0 y `contarItems` cuenta el total	Lógica (carrito)
5	`utils/carrito.test.js`	*obtenerItemsCompletos… y calcularTotal*	Une el carrito con el catálogo, descarta ids inexistentes y suma por tipo (residencial/comercial)	Lógica (compras)
6	`utils/productos.test.js`	*aplicarEdiciones, catalogoActual y productosConCreados*	Sobreescribe solo los campos editados y construye el catálogo real (base + ediciones + creados)	Lógica (CRUD) + `localStorage`
7	`components/Producto.test.jsx`	*renderiza los props… y avisa al agregar (mock)*	Renderiza **props** en el DOM y llama a `onAgregar(id)` mediante un **mock** (`vi.fn()`)	Componente (props + DOM + mock)
8	`components/Navbar.test.jsx`	*muestra los enlaces… y el total de items (prop)*	Rutas de los enlaces y el **badge** con el prop `totalItems`	Componente (props + DOM)
9	`pages/Nosotros.test.jsx`	*expande y contrae el blog…*	Cambio de **estado** (`useState`) al presionar "Ver Más" / "Ver Menos"	Componente (estado + DOM)
10	`pages/Login.test.jsx`	*valida correo, contraseña y credenciales; inicia sesión (cliente y admin) y cierra sesión*	**Formulario controlado**: campos asociados por `label` , escritura con `user-event` , errores de validación (correo, contraseña, credenciales), login exitoso de cliente y admin, y cierre de sesión	Formulario interactivo

5. Resultado de la ejecución

```
 ✓ src/utils/validaciones.test.js (2 tests)
✓ src/utils/carrito.test.js      (3 tests)
✓ src/utils/productos.test.js    (1 test)
✓ src/components/Producto.test.jsx (1 test)
✓ src/components/Navbar.test.jsx   (1 test)
✓ src/pages/Nosotros.test.jsx      (1 test)
✓ src/pages/Login.test.jsx         (1 test)

Test Files  7 passed (7)
     Tests  10 passed (10)
```

Indicador	Valor
Pruebas ejecutadas	**10**
Pruebas exitosas	**10**
Pruebas fallidas	**0**
Tiempo aproximado	~11 segundos (incluye arranque del entorno jsdom)

> Las advertencias `404: /IMAGENES/...` que pueden aparecer al ejecutar **no son errores**: en las pruebas los `<img>` apuntan a recursos que el entorno de pruebas no sirve. No afectan a los resultados ni a la cobertura (los datos del test de `<Producto />` usan `/x.png` solo para verificar el render).

---

6. Reporte de cobertura

Cobertura obtenida sobre los módulos que participan en las pruebas (provider `v8`):

Archivo	% Sentencias	% Ramas	% Funciones	% Líneas
`components/Navbar.jsx`	100	100	100	100
`components/Producto.jsx`	100	100	100	100
`datos/productos.js`	100	100	100	100
`pages/Nosotros.jsx`	100	100	100	100
`pages/Login.jsx`	100	100	100	100
`utils/carrito.js`	100	100	100	100
`utils/productos.js`	100	100	100	100
`utils/validaciones.js`	100	100	100	100
TOTAL	**100**	**100**	**100**	**100**

Resumen:

```
 Statements : 100% (426/426)
Branches   : 100% (141/141)
Functions  : 100% (30/30)
Lines      : 100% (426/426)
```

Lectura de los indicadores

- **100% de sentencias, ramas, funciones y líneas** en todos los módulos bajo prueba: cada línea y cada camino de los módulos de lógica (`utils/` , `datos/`), de los componentes `Navbar` , `Producto` y de las páginas `Nosotros` y `Login` fue ejecutado y verificado por las 10 pruebas.
- **Ramas 100%:** se cubrieron también los **caminos defensivos** (cantidades no numéricas en el carrito, respaldos `|| 0` al crear productos, mensajes alternativos de validación, formatos `email` / `password` heredados del sistema anterior) y el **flujo completo del login**: errores, credenciales válidas de cliente y administrador, y cierre de sesión.

---

7. Análisis de resultados

7.1 Cobertura de lógica vs. de interfaz

Las pruebas cubren **tres niveles complementarios**:

1. **Lógica pura (funciones de `utils/`)** — se prueban sin DOM: entradas y salidas deterministas. Es la base con mayor cobertura y la más valiosa, porque concentra las reglas de negocio (validaciones, cálculo del carrito, ediciones del catálogo → **CRUD**).
2. **Componentes y estado (React)** — se renderizan en jsdom y se interactúa con el DOM (`fireEvent.click`), verificando props, estado y eventos.
3. **Formulario interactivo (`Login`)** — con `user-event` se simula la escritura real del usuario y se verifica que el componente reacciona mostrando el error de validación.

7.2 Uso de mocks

- En la prueba **#7** se usa `vi.fn()` como **mock** del callback que el componente padre proveería. Así se verifica que `<Producto />` no solo se renderiza, sino que **notifica correctamente al hacer click**, pasando el `id` correcto. Esto desacopla el componente de la lógica del carrito.
- En las pruebas **#2** y **#6** se usa `localStorage` como dependencia controlada (se preparan datos y se limpia entre pruebas con `beforeEach(() => localStorage.clear())`), simulando la persistencia del panel admin sin backend.

7.3 Detalle de ramas cubiertas (ampliación)

Las ramas que inicialmente no se alcanzaban eran **caminos defensivos o alternativos** sin recorrer. Con **aserciones adicionales dentro de las mismas 10 pruebas** (sin aumentar su número, respetando el máximo indicado por el docente) se cubrieron todas:

Archivo	Ramas cubiertas	Cómo se ejerció
`utils/carrito.js`	`agregarItem` / `incrementarItem` con un carrito mixto (ítem que coincide + ítem que no) y `contarItems` con cantidad no numérica (`Number(item.cantidad) || 0`)	Se agregaron llamadas con arreglos de dos ítems y un caso con `cantidad: 'abc'`
`utils/productos.js`	Ternarios `ed.campo !== undefined ? … : …` de **todos** los campos editables (descripción, comercial, stock, stock crítico, categoría, imagen) y respaldos `|| 0` de productos creados sin precios/stock	Se amplió la edición guardada en `localStorage` a varios campos y se creó un segundo producto "básico"
`utils/validaciones.js`	Validaciones alternativas: nombre > 100, teléfono inválido, dirección/región/comuna faltantes en Registro, y nombre > 100, teléfono inválido, mensaje > 500 en Contacto	Se probó cada "siguiente" rama del primer-error de ambos formularios
`pages/Login.jsx`	Contraseña corta, credenciales incorrectas, login exitoso (cliente y admin), vista "Sesión iniciada", botón `Panel Admin` (solo rol Administrador) y cierre de sesión; además del formato heredado `email` / `password`	Se extendió la prueba del formulario a un **flujo completo**: errores → login cliente → logout → login admin

Este enfoque mantiene la cobertura **real y verificable** de lo efectivamente probado: no se agregaron pruebas vacías ni archivos sin ejecutar. Las **funciones y las líneas** ya estaban 100% cubiertas; la ampliación cerró las combinaciones de ramas restantes.

7.4 Alcance del reporte

El porcentaje **no** representa la totalidad de la aplicación (por ejemplo, `App.jsx`, las páginas públicas restantes y el panel de administración no figuran), sino **los módulos que las 10 pruebas unitarias ejercitan**. Se optó por medir la cobertura de lo efectivamente probado para que el indicador sea **real y verificable**, en lugar de inflarlo incluyendo archivos sin pruebas — mismo criterio que el setup anterior de Karma con `includeAllSources: false`. Ampliar el alcance implicaría agregar pruebas de integración/end-to-end.

---

8. Conclusiones

- Se implementaron **10 pruebas unitarias** con **Vitest** (el framework solicitado por el docente para esta evaluación), cumpliendo el **máximo de 10** indicado.
- Las pruebas cubren los tipos pedidos por el profesor: **componentes que validan**, **formularios interactivos**, **carrito/compras** y **CRUD del catálogo**, además de props, estado y mocks.
- El proyecto queda con **100% de sentencias, ramas, funciones y líneas** en todos los módulos bajo prueba (`utils/` , `datos/` , `Navbar` , `Producto` , `Nosotros` y `Login`), incluyendo los caminos defensivos y el flujo completo de inicio/cierre de sesión.
- La ejecución es **automatizable** (`npm test`), puede correr en **modo vigilancia** (`npm run test:watch`) y deja un **reporte HTML** navegable (`coverage/index.html`) para el equipo.

---

9. Recomendaciones / trabajo futuro

1. **Sumar pruebas de integración** de la página `Carrito` y del `App` (ruteo y estado global del carrito) para cubrir también las páginas y el panel de administración (su cobertura no figura en este documento porque el alcance es unitario).
2. **Umbral mínimo de cobertura:** configurar `coverage.thresholds` para que la integración continua falle si la cobertura baja de un mínimo acordado.
3. **Mantener la cobertura al ampliar el app:** cada página nueva (Registro, Contacto, Carrito, admin…) debería acompañarse de su prueba unitaria para conservar el nivel de calidad actual.

Anexo — Comandos y salida resumida

```
$ npm test

 ✓ src/utils/validaciones.test.js  (2 tests)
 ✓ src/utils/carrito.test.js       (3 tests)
 ✓ src/utils/productos.test.js     (1 test)
 ✓ src/components/Producto.test.jsx (1 test)
 ✓ src/components/Navbar.test.jsx   (1 test)
 ✓ src/pages/Nosotros.test.jsx      (1 test)
 ✓ src/pages/Login.test.jsx         (1 test)

 Test Files  7 passed (7)
      Tests  10 passed (10)
```

```
$ npm run test:coverage

File              | % Stmts | % Branch | % Funcs | % Lines |
------------------|---------|----------|---------|---------|
All files         |     100 |      100 |     100 |     100 |
------------------|---------|----------|---------|---------|

Statements : 100% (426/426)
Branches   : 100% (141/141)
Functions  : 100% (30/30)
Lines      : 100% (426/426)
```